02 / GUÍA DE REFERENCIA PYTHON

Herramientas NumPy para rotaciones 2D

EX29 · Funciones, entradas, salidas y ejemplos independientes

Objetivo
Relacionar las fórmulas de la Guía de Teoría con instrucciones concretas de Python. Cada función se describe indicando qué recibe, qué devuelve y cuándo se utiliza.

1. `np.deg2rad()`: grados a radianes

Entrada: número o array que representa ángulos en grados. **Salida:** número o array con los mismos ángulos en radianes. Las funciones trigonométricas de NumPy esperan radianes.

```
import numpy as np
angle_deg = 90.0
angle_rad = np.deg2rad(angle_deg)
print(angle_rad)  # approx. 1.5707963268
```

Se puede volver a grados con `np.rad2deg()`, utilizada en EX26. Recuerda: $180° = \pi$ rad.

2. `np.cos()` y `np.sin()`

Entrada: ángulo en **radianes**, escalar o array. **Salida:** coseno o seno, como número de coma flotante o array, según la entrada.

```
angle_rad = np.deg2rad(90.0)
c = np.cos(angle_rad)
s = np.sin(angle_rad)
print(c)  # about 6.12e-17, numerically close to zero
print(s)  # 1.0
```

6.12×10^{-17} no es un error geométrico: es un efecto de la precisión finita de los números `float`. No se espera igualdad exacta con cero.

3. `np.array()` y construcción de una matriz

Entrada: lista de filas. **Salida:** array NumPy de dos dimensiones. Para una matriz 2×2, su `shape` será `(2, 2)`. Ejemplo completo independiente:

```
angle = np.deg2rad(45.0)
c, s = np.cos(angle), np.sin(angle)
R = np.array([[c, -s], [s, c]])
print(R.shape)  # (2, 2)
```

Una columna describe la imagen del primer vector unitario y la otra, la imagen del segundo.

4. Operador `@`: matriz por vector

Entrada: arrays de dimensiones compatibles, por ejemplo una matriz `(2, 2)` y un vector `(2,)`. **Salida:** array `(2,)` con las componentes transformadas.

```
vector = np.array([1.0, 0.0])
new_vector = R @ vector
print(new_vector)  # approximately [0.70710678, 0.70710678]
```

`R * vector` **no** significa el mismo producto: NumPy utiliza operaciones por componentes y difusión de dimensiones (*broadcasting*).

5. `.T`: matriz transpuesta

Entrada: array de dos dimensiones. **Salida:** vista transpuesta de la matriz. En rotaciones ortogonales, `R.T` representa la transformación inversa.

```
vector_back = R.T @ new_vector
print(vector_back)  # approximately [1.0, 0.0]
```

La transposición de un array de forma `(2,)` no lo transforma en una matriz columna. Esa diferencia la estudiaremos más adelante.

6. `np.eye()`, `np.allclose()` y normas

`np.eye(2)` crea una matriz identidad 2×2. `np.allclose(a,b)` devuelve un único booleano si dos arrays son aproximadamente iguales con una tolerancia numérica razonable; no devuelve un array de booleanos como `np.isclose()`.

```
identity = np.eye(2)
print(np.allclose(R.T @ R, identity))  # True
print(np.allclose(vector_back, vector))  # True
print(np.linalg.norm(vector))           # 1.0
```

7. Resumen de elección de herramienta

Herramienta	Cuándo utilizarla
`np.deg2rad(a)`	Si el ángulo conocido está en grados.
`np.sin(a), np.cos(a)`	Para los coeficientes trigonométricos de R, con a en radianes.
`R @ v`	Para rotar activamente un vector.
`R.T @ v`	Para expresar sus componentes en ejes girados (convención del EX29).
`np.allclose()`	Para verificaciones numéricas, no igualdad exacta.
`np.linalg.norm(v)`	Para verificar la conservación de la longitud.

Errores típicos
Mezclar grados y radianes, confundir `*` con `@`, utilizar R en lugar de R^T cuando se giran los ejes y exigir que resultados de coma flotante sean exactamente cero.